

LAB FÖYÜ - 01: SQL Temelleri ve İlişkisel Modelleme

2025-2026 Eğitim-Öğretim Yılı / Bahar Dönemi

1. Haftalık Teorik Özet (Refresher)

ANSI-SPARC Mimarisi ve Veri Bağımsızlığı

Veri tabanı sistemleri, veriyi kullanıcıdan ve fiziksel depolamadan **soyutlamak** için üç seviyeli bir mimari kullanır:

- **External (Dış Seviye / View):** Kullanıcı veya uygulamanın gördüğü **özelleştirilmiş görünüm**dür.
 - Amaç: Yetki yönetimi (kim neyi görür?), karmaşıklığı azaltma.
 - Örnek: Öğrenci işleri personeli sadece **AdSoyad**, **BolumAd** alanlarını görür.
- **Conceptual (Kavramsal Seviye / Logical Schema):** Tüm kurumun veri modelidir: **tablolar, ilişkiler, kısıtlar** ve **iş kuralları** burada tanımlanır.
 - Amaç: **Doğru modelleme** (PK–FK ilişkileri, kardinaliteler, kısıtlar).
 - Örnek: **Oğrenciler(BolumID)** alanı **Bolumler(BolumID)** alanına **FK** ile bağlıdır.
- **Internal (İç Seviye / Physical Storage):** Verinin disk üzerinde nasıl tutulduğudur: **sayfa (page), blok, indeks, B-tree, dosya organizasyonu**.
 - Amaç: Performans ve depolama optimizasyonu.
 - Örnek: **OgrID** üzerinde indeks ile sorguların hızlanması.

Veri Bağımsızlığı (Data Independence):

- **Fiziksel veri bağımsızlığı:** İç seviye değişse bile (örn. indeks ekleme, disk türü) kavramsal şema değişmez.
- **Mantıksal veri bağımsızlığı:** Kavramsal şemadaki sınırlı değişimler (örn. yeni sütun) view/uygulamaları minimum etkiler.

Stratejik Sonuç: ANSI-SPARC mimarisi sayesinde veri tabanı sistemi;

- Ölçeklenebilir olur,
- Bakımı kolaylaşır,
- Güvenlik ve performans ayrı ayrı optimize edilebilir,
- Uygulama katmanından bağımsız şekilde geliştirilebilir.

İlişkisel Modelin Matematiksel Çerçevesi

İlişkisel modelde bir tablo (ilişki) R , alan kümelerinin kartezyen çarpımının bir alt kümesidir:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n = \prod_{i=1}^n D_i.$$

Burada:

- D_i : i 'inci sütunun alabileceği değerlerin kümesi (domain),
- n : sütun sayısı,
- R 'nin her bir elemanı: bir satır (tuple/kayıt).

Anahtar fikri: Veri tabanı tasarımında amaç, R üzerinde **kısıtlar** (constraints) tanımlayarak verinin **tutarlı**, **tekrarsız** ve **doğrulanabilir** kalmasını sağlamaktır.

Veri Entropisi (Data Entropy) ve Normalizasyonun Amacı

Bu derste **veri entropisi** kavramını, bilgi teorisindeki Shannon entropisinden ziyade, veri tasarımında **düzensizlik/tekrar/çıktı belirsizliği** anlamında kullanacağız:

- **Yüksek veri entropisi:** Aynı bilginin birden çok yerde tekrarlandığı, güncelleme- nin zorlaştığı, hataya açık tasarım.
- **Düşük veri entropisi:** Bilginin tek kaynaktan yönetildiği, gereksiz tekrarın azal- tıldığı, tutarlılığı güçlü tasarım.

Neden problem? (Anomaliler)

- **Ekleme anomalisi:** Bir bilgi girilebilmek için başka gereksiz bilgiler de zorunlu olur.
- **Silme anomalisi:** Bir kayıt silinince, aslında korunması gereken bilgi de kaybolur.
- **Güncelleme anomalisi:** Aynı bilgi birçok satırda olduğu için, hepsini güncellemek gerekir; biri kaçarsa tutarsızlık oluşur.

Normalizasyon (1NF–2NF–3NF): Bu anomalileri azaltarak veri entropisini düşür- meyi hedefler.

ACID: Transaction (İşlem) Güvencesi

Transaction, birden fazla SQL adımının tek bir mantıksal iş olarak ele alınmasıdır. Örnek: Banka transferi gibi, **A hesabından düş + B hesabına ekle** birlikte anlamlıdır.

ACID ilkeleri, transaction'ların güvenilir çalışmasını garanti eden temel kurallardır:

1. **Atomicity (Atomiklik)**: İşlem ya tamamen uygulanır ya hiç uygulanmaz.

Commit veya Rollback

2. **Consistency (Tutarlılık)**: İşlem öncesi ve sonrası sistem **kısıtlarla tutarlı** kalır.

- PK/FK, NOT NULL, CHECK, UNIQUE ihlal edilemez.
- **Primary Key (PK)**: Bir tablodaki her satırı benzersiz (unique) olarak tanımlayan ve NULL değer alamayan bir veya birden fazla sütundan oluşan anahtar alanıdır.
- **Foreign Key (FK)**: Bir tablodaki bir sütunun, başka bir tablonun Primary Key alanına referans vererek iki tablo arasında referansal bütünlük sağlayan anahtar alanıdır.

3. **Isolation (İzolelik)**: Eşzamanlı işlemler birbirinin **ara (incomplete) durumlarını** görmez.

- Amaç: **dirty read** (kirli okuma) gibi sorunları önlemek.

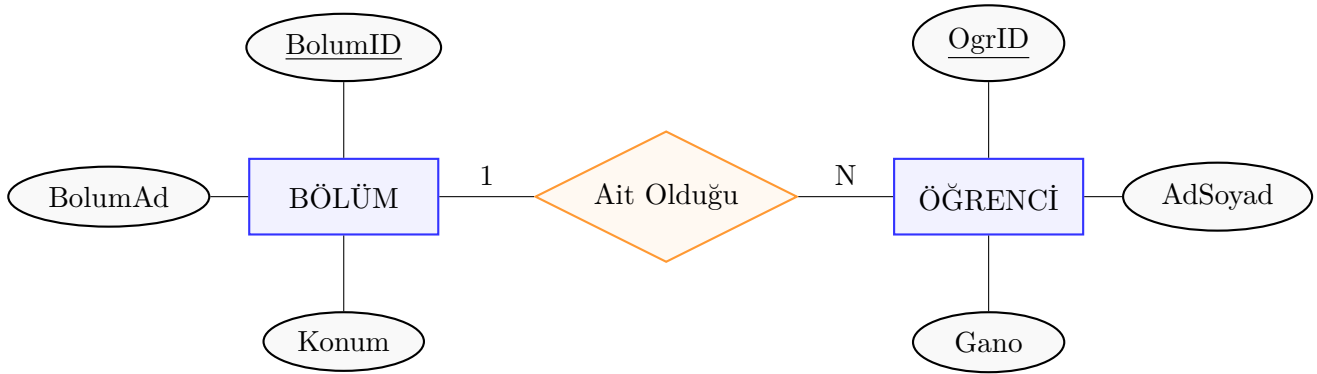
4. **Durability (Kalıcılık)**: COMMIT edilen işlem, elektrik kesilse bile kalıcıdır.

MYO-seviyesi kritik ayrım: Constraints (PK/FK/CHECK) veri doğruluğunu sağlar; **ACID** ise çoklu adımlı işlemlerin güvenilir tamamlanmasını sağlar.

2. Tasarım Kuralları ve Normalizasyon

2.1. ER → İlişkisel Dönüşüm Algoritması

- Her **Varlık** bir tabloya dönüşür.
- **Basit Öznitelikler** sütun olur. **Anahtar Öznitelikler** Primary Key (PK) olur.
- **1:N İlişkiler**: "1" tarafındaki PK, "N" tarafına Foreign Key (FK) olarak eklenir.
- **M:N İlişkiler**: Bağımsız bir "Ara Tablo" oluşturulur. Bu tablo her iki varlığın PK'larını bünyesinde toplar.



2.2. Normalizasyon: Veri Entropisinin Azaltılması

Seviye	Kural
1NF	Hücreler atomik olmalı. Tekrarlanan gruplar/çok değerli alanlar olmamalı.
2NF	1NF + Kısmi fonksiyonel bağımlılık (Partial Dependency) olmamalı.
3NF	2NF + Geçişli bağımlılık (Transitive Dependency) olmamalı. ($A \rightarrow B \rightarrow C$ durumu).

3. DDL (Data Definition Language) - Veri Tanımlama

Bu bölümde MS SQL (T-SQL) ve MySQL arasındaki ufak sözdizimi farklarını inceleyeceğiz.

3.1. Tablo Oluşturma (CREATE TABLE)

MS SQL Server (T-SQL)

```

1 CREATE TABLE Uyeler (
2     UyeID INT PRIMARY KEY,
3     Ad NVARCHAR(50) NOT NULL,
4     Soyad NVARCHAR(50),
5     Yas INT CHECK (Yas >= 18),
6     KayitTarihi DATETIME
7     DEFAULT GETDATE()
8 );
  
```

MySQL (XAMPP)

```

1 CREATE TABLE Uyeler (
2     UyeID INT PRIMARY KEY,
3     Ad VARCHAR(50) NOT NULL,
4     Soyad VARCHAR(50),
5     Yas INT,
6     KayitTarihi DATETIME
7     DEFAULT CURRENT_TIMESTAMP,
8     CONSTRAINT chk_yas CHECK (Yas >= 18)
9 );
  
```

3.2. Tabloyu Güncelleme (ALTER)

```

1 -- Yeni sütun ekleme
2 ALTER TABLE Uyeler ADD Telefon VARCHAR(15);
3
4 -- Tabloyu silme (Dikkat: Verilerle birlikte gider!)
5 DROP TABLE Uyeler;
  
```

4. DML (Data Manipulation Language) - Veri İşleme

4.1. Veri Ekleme (INSERT)

```
1 INSERT INTO Uyeler (UyeID, Ad, Soyad, Yas)
2 VALUES (1, 'Hasan', 'Oguz', 34);
3
4 -- Çoklu ıkayt ekleme (MySQL)
5 INSERT INTO Uyeler VALUES
6 (2, 'Ayse', 'Demir', 22, NOW(), '555...'),
7 (3, 'Can', 'Yilmaz', 19, NOW(), '544...');
```

4.2. İlişkisel Bütünlük ve CASCADE Mekanizması

```
1 CREATE TABLE Odunc (
2     IslemID INT PRIMARY KEY,
3     KitapID INT,
4     UyeID INT,
5     FOREIGN KEY (UyeID) REFERENCES Uyeler(UyeID)
6     ON DELETE CASCADE -- Uye silinirse odunc kaydi da silinir.
7     ON UPDATE CASCADE -- Uye ID degisirse otomatik guncellenir.
8 );
```

Kritik Uyarı

ON DELETE RESTRICT (Varsayılan): Eğer ilişkili bir ödünç kaydı varsa, sistem üye kaydını sildirmez. Bu, veri bütünlüğünü korumak için en güvenli yoldur.

5. Laboratuvar Uygulama Görevleri

Laboratuvar: D207 / MS SQL Server & XAMPP

- Görev 1:** Kendi adınıza bir veri tabanı oluşturun (CREATE DATABASE OgresciNo_AdSoyad).
- Görev 2:** Aşağıdaki ER diyagramına göre "Bölümler" ve "Öğrenciler" tablolarını oluşturun. Bölümler ve Öğrenciler arasında 1:N ilişki kurun.
- Görev 3:** CHECK kısıtı kullanarak, öğrenci notunun 0 ile 100 arasında olmasını garanti altına alın.
- Görev 4:** ON DELETE SET NULL kısıtını test edin. Bir bölümü sildiğinizde, o bölüme ait öğrencilerin "BolumID" alanının boşaldığını gözlemleyin.
- Görev 5 (Challenge):** Aynı tabloyu hem SSMS (MS SQL) hem de phpMyAdmin (MySQL) üzerinde oluşturun. NVARCHAR ve VARCHAR arasındaki farkları not alın.

Haftalık Kontrol Listesi (Checklist)

- ☐ ANSI-SPARC mimarisini ve katmanlarını anladım.
- ☐ PK ve FK arasındaki "Referansal Bütünlük" bağıı kurabiliyorum.
- ☐ ACID ilkelerinin önemini ve Transaction mantığını kavradım.
- ☐ DDL ve DML komutları arasındaki farkı biliyorum.

"Veri, mühendislik disipliniyle işlendiğinde anlam kazanır."

Dr. Öğr. Üyesi Hasan Oğuz

Ek: Çözüm Anahtarı (Lab-01)

```

1  /* MBLP136 - Lab 01 Çözüm Anahtarı
2     Platform: MS SQL Server (T-SQL) & MySQL (XAMPP)
3  */
4
5  -- GÖREV 1: Veri Tabanı Oluşturma
6  CREATE DATABASE MBLP136_HasanOguz;
7  GO
8  USE MBLP136_HasanOguz;
9
10 -- GÖREV 2: Bölümler Tablosu (Ebeveyn - Parent)
11 CREATE TABLE Boluimler (
12     BolumID INT PRIMARY KEY, -- Anahtar Öznitelik
13     BolumAd VARCHAR(100) NOT NULL,
14     Konum VARCHAR(50)
15 );
16
17 -- GÖREV 2 & 3: Öğrenciler Tablosu (Çocuk - Child)
18 CREATE TABLE Ogrenciler (
19     OgrID INT PRIMARY KEY,
20     AdSoyad VARCHAR(100) NOT NULL,
21     -- GÖREV 3: CHECK Kisiti (Notun 0-100 aras 1olmas)
22     Gano DECIMAL(3,2) CONSTRAINT chk_gano CHECK (Gano >= 0 AND Gano <= 4.00),
23     BolumID INT,
24
25     -- GÖREV 4: ON DELETE SET NULL 1Mekanizmas
26     -- Bölüm silindiginde öğrenci kaydi kalir, ancak BolumID alan 1NULL olur.
27     CONSTRAINT fk_bolum FOREIGN KEY (BolumID)
28     REFERENCES Boluimler(BolumID)
29     ON DELETE SET NULL
30     ON UPDATE CASCADE
31 );
32
33 -- TEST VERILERI EKLEME
34 INSERT INTO Boluimler VALUES (1, 'Elektronik ve Haberlesme', 'C-Blok');
35 INSERT INTO Boluimler VALUES (2, 'Bilgisayar Programciligi', 'D-Blok');
36
37 INSERT INTO Ogrenciler (OgrID, AdSoyad, Gano, BolumID)
38 VALUES (2601, 'Ahmet Yilmaz', 3.50, 1);
39 INSERT INTO Ogrenciler (OgrID, AdSoyad, Gano, BolumID)
40 VALUES (2602, 'Ayse Demir', 2.80, 1);
41
42 -- GÖREV 3 TEST: Hatali Veri Girisi (Fails due to CHECK constraint)
43 -- INSERT INTO Ogrenciler VALUES (2603, 'Hatali Kayit', 5.00, 2);
44
45 -- GÖREV 4 TEST: Referansal Bütünlük (SET NULL denemesi)
46 DELETE FROM Boluimler WHERE BolumID = 1;
47
48 -- SONUCU GÖZLEMLEME
49 SELECT * FROM Ogrenciler;
50 -- Ç11ktda Ahmet ve Ayse'nin BolumID 11alannn NULL olduđu görülecektir.

```